FundFlow SaaS — Initial Application Design

Overview

A multi-tenant SaaS application for managing family funds, built with Laravel 12, Filament v5, and `stancl/tenancy` v3. Each family fund operates as an isolated tenant with its own database, accessible via a unique subdomain (e.g., `samman.fundflow-saas.osamman.com`).

Architecture

```
flowchart TB
    subgraph central ["Central Domain (fundflow-saas.osamman.com)"]
        centralAdmin["/admin - Central Admin Panel"]
        centralDB[(Central Database)]
    end
    subgraph tenantA ["Tenant Domain (samman.fundflow-saas.osamman.com)"]
        landing["/ - Public Landing Page"]
        adminPanel["/manage - Fund Admin Panel"]
        memberPortal["/portal - Member Portal"]
        tenantDB[(Tenant Database)]
    end
    centralAdmin --> centralDB
    landing --> tenantDB
    adminPanel --> tenantDB
    memberPortal --> tenantDB
```

Multi-Tenancy

- **Package**: `stancl/tenancy` v3
- **Strategy**: Database-per-tenant with domain-based identification
- **Bootstrappers**: Database, Cache, Filesystem, Queue, Redis
- **Tenant creation**: Automatic domain creation (`{tenant_id}.fundflow-saas.osamman.com`), database migration, and seeding via `TenantCreated` event pipeline

Domain Model

Financial Structure

Each fund has two **master accounts** owned by the fund itself: - **Master Cash**: Entry account where incoming cash is received - **Master Fund**: Reflects the total accumulated member fund savings

Each member has two **personal accounts**: - **Cash Account**: Where their incoming funds land - **Fund Account**: Long-term savings from posted contributions

Double-Entry Accounting Flow

```
flowchart LR
    MasterCash["Master Cash"]
    MemberCash["Member Cash"]
    MemberFund["Member Fund"]
    MasterFund["Master Fund"]

    MasterCash -->|"allocate"| MemberCash
    MemberCash -->|"post"| MemberFund
    MasterFund -.->|"reflects total"| MemberFund
```

Contribution posting flow: 1. Credit Master Cash (money received) 2. Transfer from Master Cash to Member Cash (allocated) 3. Transfer from Member Cash to Member Fund (posted) 4. Credit Master Fund (reflects total member funds)

Loan disbursement flow: 1. Debit Master Fund (funded from collective) 2. Credit Member Cash (member receives funds)

Member Relationships

Members can have a **parent-dependent** relationship: - A parent member is responsible for making monthly contributions for their dependents - The parent defines the monthly contribution amount for each dependent - Dependents are linked via `parent_member_id` on the members table

Loan Eligibility

Members can apply for loans if they meet all conditions: - Minimum 12 months of active membership - Status is `active` - No missed contributions in the last 3 months - No currently active loans (disbursed or repaying)

Database Schema (Tenant)

Tables

Table	Purpose
`users`	Authenticatable users with `is_admin` flag
`members`	Fund members with parent/dependent hierarchy
`accounts`	Cash and fund accounts (master + per-member)
`transactions`	Double-entry transaction ledger
`contributions`	Monthly contribution records
`loans`	Loan applications and lifecycle
`loan_repayments`	Individual loan repayment records
`membership_applications`	Public enrollment submissions

Key Relationships

```
erDiagram
    User ||--o| Member : "has one"
    Member ||--o{ Member : "parent/dependents"
    Member ||--o{ Account : "has many"
    Member ||--o{ Contribution : "has many"
    Member ||--o{ Loan : "has many"
    Account ||--o{ Transaction : "has many"
    Loan ||--o{ LoanRepayment : "has many"
```

Service Layer

Business logic is encapsulated in three service classes:

`AccountingService`

- `credit()` / `debit()` — Record transactions with running balance
- `transfer()` — Atomic debit-from + credit-to in a DB transaction
- `createMemberAccounts()` — Create cash + fund accounts for a new member

`ContributionService`

- `recordContribution()` — Create a pending contribution record
- `postContribution()` — Execute the full accounting flow (Master Cash → Member Cash → Member Fund → Master Fund)
- `generateMonthlyContributions()` — Batch-create pending contributions for all active members

`LoanService`

- `checkEligibility()` — Validate loan eligibility requirements
- `approveLoan()` / `disburseLoan()` — Loan lifecycle management with accounting entries
- `recordRepayment()` — Process loan repayments

Filament Panels

Central Admin Panel (`/admin`)

- Manages tenants, plans, subscriptions, users
- Uses `FilamentShield` for role-based access control
- Color: Amber

Tenant Admin Panel (`/manage`)

- Full CRUD for members, accounts, contributions, loans, membership applications
- Dashboard with fund overview stats (master balances, member count, pending contributions, active loans)
- Color: Emerald
- Auth guard: `tenant`

Member Portal (`/portal`) — See `docs/member-portal.md`

Public Tenant Pages

Landing Page (`/`)

- Responsive marketing page with Tailwind CSS v4
- Sections: Hero, Features (6 cards), How It Works (4 steps), Enrollment Form
- Livewire-powered membership application form

Authentication

- Two auth guards: `web` (central) and `tenant` (tenant users)
- Early tenancy initialization via `InitializeTenancyByDomainEarly` global middleware to ensure database sessions and CSRF work correctly with tenant databases

Tech Stack

Component	Technology
Framework	Laravel 12
Admin Panels	Filament v5
Multi-tenancy	stancl/tenancy v3
Frontend	Tailwind CSS v4, Livewire v4
Testing	Pest v4
Sessions	Database (tenant-scoped)
PHP	8.4
Node.js	22 LTS

File Structure (Tenant-Specific)

```
app/
  Filament/
      Tenant/Resources/          # Admin panel resources
          Members/
          Accounts/
          Contributions/
          Loans/
          MembershipApplications/
      Tenant/Widgets/            # Admin dashboard widgets
      Member/Resources/          # Member portal resources
           MyAccounts/
           MyContributions/
           MyLoans/
  Models/Tenant/                 # Tenant-scoped Eloquent models
  Services/                      # Business logic services
  Livewire/Tenant/               # Livewire components
  Providers/Filament/            # Panel providers

database/
  migrations/tenant/             # Tenant database migrations
  factories/Tenant/              # Tenant model factories
  seeders/Tenant/                # Tenant database seeders

resources/views/
  tenant/landing.blade.php       # Public landing page
  livewire/tenant/               # Livewire views

tests/Feature/Tenant/              # Tenant feature tests
```